

ADNEKO · KIRA SOFTWARE

Documento de cierre técnico

Sí existe software real y una arquitectura ejecutable de KIRA. El repositorio contiene el runtime, schema PostgreSQL 32, API, servicios Execution M1, reconciliación, cancelación, verificación y tres flujos E2E definidos.

Estado de referencia del branch: HEAD 81b123c065cd6ce48aec40596a0aa65f5ebe47cd · PR #1 OPEN/DRAFT/MERGEABLE ·
819 commits · 147 archivos · Schema 32

Pregunta	Respuesta actual
¿Hay software real?	Sí. FastAPI, dominio KIRA, PostgreSQL, Supabase Auth, Execution M1, dispatch/replay, reconciliación, cancelación, verificación, migraciones y tooling de certificación existen en el repositorio.
¿Hay algo que se pueda ejecutar?	Sí. Runtime FastAPI, migraciones, bootstrap de identidad, ingestión, expiración de clarificaciones y los servicios internos de Execution M1 están implementados. Hay 3 recorridos E2E PostgreSQL de presentación definidos.
¿Puedo decir hoy «lo vi funcionando»?	NO TODAVÍA con evidencia final. Falta ejecutar el certifier en una PostgreSQL desechable y obtener el mensaje SUCCESS. GitHub Actions no ha dado una señal válida porque los jobs fallan antes de crear steps/logs.
¿Está desplegado y navegable en producción?	NO VERIFICADO. Existe render.yaml y la composición de producción, pero no se ha verificado en este cierre un servicio Render/Supabase listo y accesible.
¿Existe API pública para crear Execution?	NO, deliberadamente. ExecutionCreationService exige un AuthorizationBinding real. El API público no fabrica autoridad a partir de datos enviados por el cliente.
¿NEXUS real está conectado por red?	NO. La integración, contratos, estados, replay y reconciliación están implementados, pero el transporte NEXUS real permanece opcional/no cableado al runtime público.
¿Esto es hardware?	NO. Este paquete corresponde a KIRA SOFTWARE. BOM físico, cableado eléctrico, I/O físico y montaje mecánico no aplican aquí.

KIRA FASE 1 no es un mockup. El valor desarrollado está en el software ejecutable, el modelo de autoridad, la persistencia durable, la trazabilidad, el multi-tenant, la integración NEXUS controlada, la recuperación de incertidumbre y la certificación E2E.

Qué está hecho

Componente	Estado	Qué significa
Runtime FastAPI	IMPLEMENTADO	Entrypoint productivo, middleware, request-size limit, health/live, health/ready.
PostgreSQL	IMPLEMENTADO	Fuente durable de verdad; schema requerido 32; RLS/tenant-scoped repositories.
Supabase Auth	IMPLEMENTADO	Verificación bearer + resolución de principal/membership KIRA.
Commitments API	IMPLEMENTADO	POST/GET/PATCH con permisos y persistencia.
Execution read API	IMPLEMENTADO	GET /v1/executions/{execution_id}; expone estado/provenance sin grant/payload hash.
Execution creation interna	IMPLEMENTADO	ExecutionCreationService con AuthorizationBinding validado; atomicidad, idempotencia y concurrency recovery.
Dispatch a NEXUS	IMPLEMENTADO EN CORE	Boundary, SEND_INTENT_RECORDED, ACK/reject/uncertain, replay seguro. Transporte real no cableado.
Resultado terminal NEXUS	IMPLEMENTADO	SUCCEEDED/FAILED/REJECTED con journal, provenance, idempotencia y fail-closed corruption checks.
Reconciliación	IMPLEMENTADO	AUTHORITY_UNCERTAIN → RECONCILIATION_REQUIRED → snapshot autoritativo; redispatch ciego bloqueado.
Cancelación	IMPLEMENTADO	REQUESTED → SEND_INTENT →

		ACCEPTED/CONFIRMED/REJECTED/TOO_LATE; Execution + request coherentes.
Verificación	IMPLEMENTADO	SUCCEEDED separado de VERIFIED; evidencia durable e idempotente.
Migraciones	IMPLEMENTADO	Catálogo hasta schema 32; runtime READY solo si CURRENT.
Certifier de presentación	IMPLEMENTADO	ruff → mypy → pytest completo → 3 flujos E2E; SUCCESS solo si todos los gates pasan.
Workers de reminders/outbox/notifications	PARCIAL	Repositorios/servicios existen; faltan workers ejecutables y provider de notificación.
Extraction worker	PARCIAL	Pipeline interno existe; faltan adapters externos reales.
Deploy externo verificado	PENDIENTE	Blueprint Render existe; no se ha demostrado un runtime externo READY en este cierre.

Qué puede demostrarse materialmente

Hay tres recorridos críticos definidos como certificación PostgreSQL real. No son diagramas: son tests de integración que usan los servicios productivos y restauran el estado final desde la base.

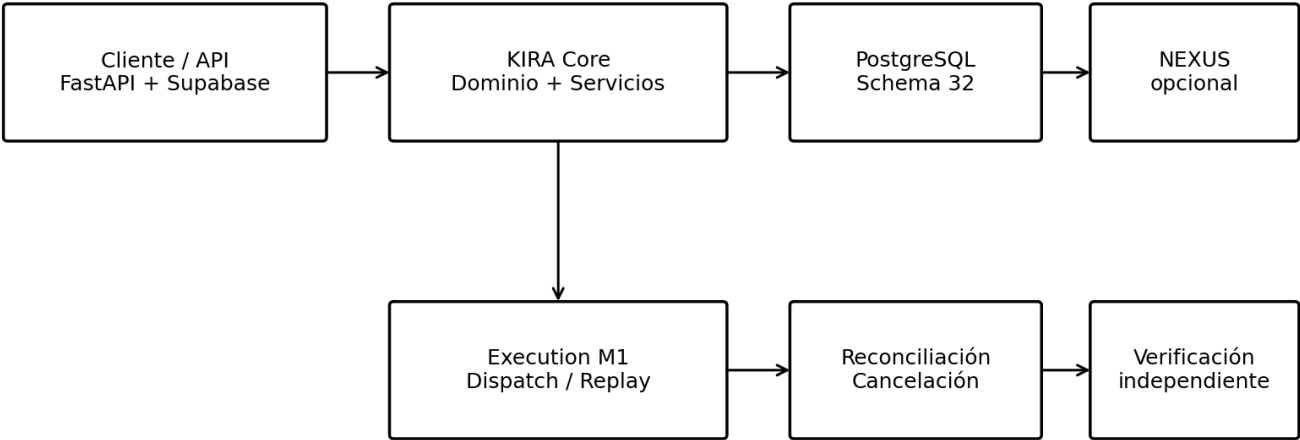
Flujo	Recorrido	Evidencia esperada
SUCCESS + VERIFICATION	CREATED → SUBMITTING → PREPARED → SEND_INTENT → ACCEPTED → RUNNING → SUCCEEDED → VERIFIED	Execution restaurado desde PostgreSQL + journal contiguo de 8 eventos.
INCERTIDUMBRE + RECOVERY	SEND_INTENT → AUTHORITY_UNCERTAIN → RECONCILIATION_REQUIRED → NEXUS_RECONCILED → SUCCEEDED	Se demuestra que no hay redispatch ciego y que el snapshot autoritativo cierra la ejecución.
CANCELACIÓN	RUNNING → CANCELLATION_REQUESTED → SEND_INTENT → ACCEPTED → CONFIRMED → CANCELLED	Execution + CancellationRequest restaurados; replay exacto no duplica evento.

Comando único de certificación:

```
export KIRA_TEST_DATABASE_URL="postgresql+psycopg://..."
export KIRA_EXECUTION_DEMO_ALLOW_DATABASE_RESET=1
python tools/certify_execution_demo.py
```

Arquitectura

KIRA SOFTWARE FASE 1 — ARQUITECTURA REAL DEL CIERRE



Nota: la integración NEXUS está modelada y endurecida; el transporte real permanece opcional/no cableado al runtime público.

Paquete de construcción

Requisito industrial	Equivalente KIRA SOFTWARE	Estado FASE 1
BOM	SBOM/dependencias: Python >=3.13,<3.14; FastAPI; SQLAlchemy; Alembic; psycopg; pydantic; uvicorn; pytest/ruff/mypy.	Disponible en pyproject.toml.
Arquitectura	API → Application services → Domain → PostgreSQL; Supabase identity; NEXUS opcional.	Documentada + implementada.
Cableado	Conexiones lógicas/env/DSN/auth/transport adapters.	Runtime DB/Auth implementado; NEXUS real pendiente.
I/O	HTTP /health/*, /v1/commitments, /v1/clarifications, /v1/executions/{id}; contratos NEXUS internos.	HTTP productivo parcial + Execution read.
Montaje	Composition root, create_app, runtime, UoW, repositories.	Implementado.
Configuración	KIRA_DATABASE_URL, KIRA_SUPABASE_URL, KIRA_SUPABASE_PUBLISHABLE_KEY, KIRA_TENANT_SLUG.	Definido.
Software	Core KIRA + Execution M1 + migrations + tooling.	Implementado.
FAT	ruff + mypy + pytest + 3 E2E PostgreSQL críticos.	Runner implementado; ejecución SUCCESS pendiente.
Puesta en marcha	Migrar schema 32 → bootstrap identity → uvicorn → health → operación.	Procedimiento documentado.
Criterio aceptación	Schema CURRENT, READY, auth correcta, E2E SUCCESS, no redispatch, replay idempotente.	Definido.

Qué falta

- 1. Preparar una PostgreSQL DESECHABLE: Nunca usar producción. El certifier resetea el schema public.
- 2. Configurar KIRA_TEST_DATABASE_URL: Debe ser postgresql+psycopg:// con usuario, host y nombre de base explícitos.
- 3. Habilitar el reset de certificación: KIRA_EXECUTION_DEMO_ALLOW_DATABASE_RESET=1.
- 4. Ejecutar python tools/certify_execution_demo.py: Debe superar ruff, mypy, pytest completo y los 3 E2E.
- 5. Guardar el receipt SUCCESS: Este es el primer punto en que puede decir «la FASE 1 pasó la certificación técnica en PostgreSQL».
- 6. Para demo por navegador: desplegar runtime: Aplicar migraciones schema 32, configurar Supabase, bootstrap identity y arrancar kira.api.server:create_app.
- 7. Comprobar /health/live y /health/ready: READY debe aparecer solo con schema CURRENT.
- 8. Probar superficie pública: Crear/leer commitment y leer un Execution ya durable mediante GET /v1/executions/{id}.
- 9. Para creación Execution pública real: Conectar una fuente real de AuthorizationBinding. No debe fabricarse desde inputs del cliente.
- 10. Para NEXUS live real: Conectar el transport/adaptador externo y repetir los mismos criterios de autoridad/replay contra ese sistema.

Criterios

Criterio	PASS cuando...	Hoy
Código	ruff y mypy terminan 0.	Runner preparado; resultado final pendiente.
Suite	pytest completo termina 0.	Runner preparado; resultado final pendiente.
DB	Migrations 1..32 aplicadas y schema CURRENT.	Contrato y migrations disponibles.
Happy path	E2E termina SUCCEEDED + VERIFIED.	Flujo definido; ejecución final pendiente.
Uncertainty	No redispatch; reconciliación termina durablemente.	Flujo definido; ejecución final pendiente.
Cancellation	Execution CANCELLED + request CONFIRMED y replay idempotente.	Flujo definido; ejecución final pendiente.
Runtime	/health/ready = READY.	Composición implementada; deployment externo no verificado.
Auth	Supabase user → KIRA principal/membership/permissions.	Implementado; identidad real demo no verificada.
Execution public creation	AuthorizationBinding proviene de fuente confiable.	PENDIENTE / fuera de API actual.
NEXUS live	Transport real entrega respuestas autoritativas y mismas invariantes se preservan.	PENDIENTE.

KIRA es un software backend real en cierre v1: runtime FastAPI, autenticación Supabase, persistencia PostgreSQL multi-tenant, schema 32, Execution M1 con dispatch durable, replay, recuperación de incertidumbre, cancelación y verificación independiente. Los tres recorridos críticos de Execution están codificados como certificación E2E PostgreSQL

Archivos que constituyen la evidencia de esta fase

Archivo	Función
README.md	Estado oficial, boundary de producto, deployment, migración, bootstrap y certificación.
tools/certify_execution_demo.py	Runner único que solo imprime SUCCESS después de todos los gates.
tests/integration/test_execution_repository_postgresql_certification.py	Certificación PostgreSQL y los 3 E2E críticos.
tests/infrastructure/test_execution_demo_certifier.py	Pruebas del propio certifier y sus protecciones.
kira/api/server.py	Entrypoint productivo.
kira/api/runtime.py / composition.py	Composición runtime + readiness gate.
kira/application/execution_creation_service.py	Creación interna autorizada e idempotente.
kira/application/execution_transaction_service.py	Transiciones y journal durable.
kira/application/nexus_dispatch.py	Boundary NEXUS: no retry ciego, validación de responses.
kira/application/nexus_observation_service.py	Resultados terminales y cancelación autoritativa.
kira/application/execution_reconciliation_service.py	Recuperación de AUTHORITY_UNCERTAIN.
kira/application/cancellation_transaction_service.py	Lifecycle durable de cancelación.
kira/application/execution_verification_service.py	VERIFIED independiente de SUCCEEDED.
kira/infrastructure/database/execution_repository.py	Persistencia/replay/journal PostgreSQL.
migrations/0001..0032	Construcción reproducible del schema.
render.yaml	Blueprint de demo/cierre; auto-deploy deshabilitado.